

CP Bootcamp 2026 — Day 15

Theme: Dynamic Programming Foundation + Problem Solving Strategy

Main Goal:

আজকে তুমি CP-এর সবচেয়ে powerful এবং challenging technique-এর foundation শিখবে
— **Dynamic Programming (DP)**।

DP প্রথমে কঠিন মনে হয় কারণ এটা নতুন কোনো algorithm না, বরং **problem solving mindset**।

আজকের মূল চিন্তা:

Brute Force

↓

Repeated Work Detect

↓

Save Previous Result

↓

Optimize

Estimated Time: 6–7 hours



Day 15 Chapter Map

Day 15

- Chapter 1 – Dynamic Programming Mental Model
- Chapter 2 – Recursion vs DP
- Chapter 3 – Memoization (Top Down DP)
- Chapter 4 – Tabulation (Bottom Up DP)
- Chapter 5 – Classic DP Problems
- Chapter 6 – State & Transition Thinking
- Chapter 7 – Space Optimization
- Chapter 8 – DP Problem Recognition
- Chapter 9 – Common Bugs & Mistakes
- Chapter 10 – Day 15 Assignment & Practice

Chapter 1 — Dynamic Programming Mental Model

Dynamic Programming কী?

Dynamic Programming হলো:

একই subproblem বারবার solve না করে result store করে রাখা।

Example:

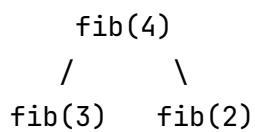
Fibonacci:

Formula:

$$\begin{aligned} &[\\ &F(n)=F(n-1)+F(n-2) \\ &] \end{aligned}$$

Normal Recursion:

```
id="fib01"  
fib(5)
```



অনেক calculation repeat হয়।

Example:

```
fib(3)
```

একাধিকবার calculate হচ্ছে।

Problem:

Repeated Work

Solution:

Store Answer

এটাই DP।

DP এর মূল Idea

Problem

↓

Break into Smaller Problems

↓

Are they repeated?

↓

Save Result

↓

Reuse

Chapter 2 — Recursion vs DP

Recursion

শুধু solve করে।

Example:

```
fib(n-1)
```

```
fib(n-2)
```

কিন্তু result রাখে না।

DP

Solve করে + store করে।

Example:

```
dp[5] = 5
```

পরেরবার দরকার হলে:

আগে থেকে আছে

Fibonacci Example

Recursion

```
int fib(int n)
{
    if(n ≤ 1)
        return n;

    return fib(n-1)+fib(n-2);
}
```

Complexity:

$O(2^n)$

খুব slow।

DP Version

Store:

dp[0]

dp[1]

dp[2]

dp[3]

Complexity:

$O(n)$

Chapter 3 — Memoization (Top Down DP)

Memoization মানে:

Recursion + Cache

Mental Model:

Call Function

↓

Already Answer আছে?

↓

Yes → Return

No → Calculate + Store

Example:

```
int dp[100];

int fib(int n)
{
    if(n ≤ 1)
        return n;

    if(dp[n] ≠ -1)
        return dp[n];

    return dp[n]=fib(n-1)+fib(n-2);
}
```

এখানে:

dp[n]

হলো memory।

Dry Run

fib(5)

First:

fib(5)

Need:

```
fib(4)
fib(3)
```

Calculate করে store:

```
dp[3]=2
```

```
dp[4]=3
```

```
dp[5]=5
```

আবার fib(3) লাগলে:

আগেই আছে:

```
dp[3]=2
```

Direct return।

Chapter 4 — Tabulation (Bottom Up DP)

Tabulation:

ছোট থেকে বড় answer build করা।

Example Fibonacci:

Need:

`fib(5)`

Start:

`dp[0]=0`

`dp[1]=1`

Build:

`dp[2]=dp[1]+dp[0]`

`dp[3]=dp[2]+dp[1]`

`dp[4]=dp[3]+dp[2]`

`dp[5]=dp[4]+dp[3]`

Code:

```

int fib(int n)
{
    int dp[100];

    dp[0]=0;
    dp[1]=1;

    for(int i=2;i<=n;i++)
    {
        dp[i]=dp[i-1]+dp[i-2];
    }

    return dp[n];
}

```

Memoization vs Tabulation

Memoization	Tabulation
Top Down	Bottom Up
Recursion	Loop
Easy transition	Usually faster
Stack ব্যবহার করে	No recursion

Chapter 5 — Classic DP Problems

Problem 1 — Fibonacci

Pattern:

Previous Results

Problem 2 — Climbing Stairs

Problem:

তুমি 1 বা 2 step করে উঠতে পারো।

Example:

$n = 5$

Ways?

Thinking:

শেষ step:

Either:

আগে 4 এ ছিলে

বা:

আগে 3 এ ছিলে

So:

$ways(n) = ways(n-1) + ways(n-2)$

Same Fibonacci pattern।

Problem 3 — 0/1 Knapsack Idea

Problem:

Bag capacity:

10 kg

Items:

Weight + Value

Maximum value নিতে হবে।

Decision:

প্রতিটি item:

Take

or

Don't Take

Pattern:

Choice

↓

State

↓

Transition

Problem 4 — Maximum Subarray Sum (Kadane's Algorithm)

Array:

-2 1 -3 4 -1 2 1

Need:

Maximum continuous sum

Idea:

Current sum:

Continue?

or

Start New?

Formula:

```
current=max(arr[i],current+arr[i])
```

Complexity:

$O(n)$

Chapter 6 — State & Transition Thinking

DP-এর সবচেয়ে important part।

State

State মানে:

কোন information ধরে answer represent করছি।

Example:

Fibonacci:

$dp[n]$

মানে:

n position-এর answer

Climbing stairs:

$dp[i]$

মানে:

i step পর্যন্ত কয়ভাবে যাওয়া যায়

Transition

এক state থেকে next state।

Example:

Fibonacci:

$dp[n] = dp[n-1] + dp[n-2]$

DP চিন্তার Formula:

State

+

Transition

+

Base Case

Chapter 7 — Space Optimization

অনেক DP-তে পুরো array দরকার হয় না।

Example:

Fibonacci:

Need:

`dp[i-1]`

`dp[i-2]`

শুধু দুইটা value লাগে।

Instead:

prev2

prev1

current

Memory:

Before:

$O(n)$

After:

$O(1)$

Chapter 8 — DP Problem Recognition

Problem দেখলে চিনবে:

Pattern 1

"Number of ways"

Example:

কতভাবে করা যায়?

Think:

DP

Pattern 2

"Maximum / Minimum"

Example:

Maximum profit

Minimum cost

Think:

DP

Pattern 3

"Choose or skip"

Example:

Take item / leave item

Think:

DP

Pattern 4

"Previous states"

Example:

Current answer depends on previous answers।

Think:

DP

Chapter 9 — Common Bugs & Mistakes

Bug 1 — Wrong Base Case

Example:

Fibonacci:

```
if(n==1)
```

কিন্তু $n=0$ handle করা হয়নি।

Bug 2 — Wrong State

DP-এর সবচেয়ে বড় mistake:

ভুল information store করা।

Bug 3 — Transition ভুল

Example:

Wrong:

$$dp[n] = dp[n-1]$$

কিন্তু দরকার:

$$dp[n-1] + dp[n-2]$$

Bug 4 — Initialize ভুল

Memoization:

Need:

$$dp[i] = -1$$

না হলে:

default value সমস্যা করবে।

Bug 5 — Over Complicate

সব problem-এ DP লাগবে না।

আগে দেখো:

Pattern আছে কিনা

Chapter 10 — Day 15 Assignment

Part A — Basic DP

Implement:

1. Fibonacci using recursion
2. Fibonacci using memoization
3. Fibonacci using tabulation
4. Fibonacci space optimized

Part B — DP Problems

Solve:

1. Climbing Stairs
2. Maximum Subarray Sum
3. Minimum Cost Path
4. Coin Change (basic idea)

Part C — State Writing Practice

প্রতিটি problem-এর জন্য লিখবে:

Example:

Climbing Stairs

State:

$dp[i]$

Meaning:

i step পর্যন্ত ways

Transition:

$dp[i] = dp[i-1] + dp[i-2]$

Base:

$dp[0]=1$

$dp[1]=1$

Part D — Pattern Recognition

Problem:

কতভাবে N পর্যন্ত যাওয়া যায়?

Pattern:

?

Problem:

Maximum profit বের করতে হবে।

Pattern:

?

Problem:

প্রতিটি item নেবো অথবা নেবো না।

Pattern:

?



Glossary Update

Add:

Dynamic Programming

DP State

DP Transition

Base State

Memoization

Tabulation

Top Down DP

Bottom Up DP

Overlapping Subproblem

Optimal Substructure

Space Optimization

State Compression



Pattern Library Update

Add:

Dynamic Programming Pattern

Recognition:

- Previous answer needed
- Repeated subproblem
- Maximum/Minimum
- Number of ways

Mental Model:

Define State

↓

Find Transition

↓

Set Base Case

↓

Optimize Memory

Take or Skip Pattern

Recognition:

Each item has choice.

Mental Model:

Choose

or

Don't Choose

Applications:

- Knapsack
- Subsequence
- Combination problems



Day 15 Completion Checklist

- [] DP কেন দরকার বুঝি
- [] Recursion vs DP difference বুঝি
- [] Memoization বুঝি
- [] Tabulation বুঝি
- [] DP State define করতে পারি
- [] Transition লিখতে পারি
- [] Fibonacci DP করতে পারি
- [] Climbing Stairs বুঝি
- [] Kadane's Algorithm বুঝি
- [] Take/Skip চিনতে পারি
- [] Space optimization বুঝি



Day 15 Final Mental Model

Problem



Can I break into smaller problems?



Are they repeated?



Define State



Find Transition



Store Result



Optimize

Day 15 শেষে:

Day 1-5
Pattern Foundation

+

Day 6
Functions

+

Day 7
Searching

+

Day 8
Sorting

+

Day 9
Optimization

+

Day 10
Recursion

+

Day 11
Stack Queue

+

Day 12
Linked List + Hashing

+

Day 13

Trees

+

Day 14

Graphs

+

Day 15

Dynamic Programming

পরের:

Day 16 — Final Contest Preparation + CP Strategy + Full Revision

এটা হবে Bootcamp-এর শেষ দিন। 🏁